
Lr
SQLite
id | filename | capture_date | rating | sync_state
1001 | DSC_8324.NEF | 2024-10-26 | 5 | synced
1002 | IMG_4178.CR3 | 2024-11-02 | 4 | synced
1003 | PXL_2981.DNG | 2024-11-10 | 5 | synced
1004 | IMG_6607.ARW | 2024-11-18 | 4 | synced
1005 | DSC_7712.NEF | 2024-12-03 | 5 | synced
INDEX
INDEX
INDEX

FirstPair Lightroom Repair

A practical cookbook for catalogs, previews and Sync

FirstPair contributors

2026-11-10

Version 1.1.0-ebc516f2

Source commit ebc516f21e39e0272a42304bee009b5c475cc5aa

First Pair Press
firstpair.press

Contents

A Lightroom repair cookbook 2
What has actually been repaired 3
Which parts are supplied as code 5
Where it fits 5
Stage and qualify the public installer 6
Choose the problem before choosing the repair 7
Establish a recovery boundary 7
Get an observation that answers one question 8
A read-only check changes the closed catalog's sidecars 9
Verify a completed receiver without replaying its initial transfer 9
Repair locations without reimporting 10
Find an actually missing lookup index 10
Diagnose SQL affinity before adding an index 11
Reduce large-map work without sharing mutable state 12
Drain and roll back a map transport normally 13
Prevent format-only capture-date requeue 14
Withdraw proved redundant metadata work through its owner 14
Repair folder admission for download workers 15
Renew bounded fixes after launch 15
Separate Sync stages and counts 16
Verify a Smart Preview at the level you need 17
Isolate a stalled Smart Preview batch 17
Keep a bulk preview build cooperative 19
Reuse offline converter-built Smart Previews 19
Map and clone Classic previews into Cloud precisely 20
Test offline rendering and editing without overstating Undo 21
Recover exact settings without resetting edits 21
Recover a selected proxy-missing upload 22
Recover verification after a successful native action 22
Classify identity before merging or deleting 23
Route native fingerprints by format 24
Prove a backup holds every photo before deleting it 25
Replace a damaged original with its intact conversion 26
Browse and rate with the originals offline 27
Reconcile catalog, XMP, ratings and dates 28
Start with supported performance maintenance 28
A receipt you can reuse 28

A Lightroom repair cookbook

This cookbook explains repairs and diagnostic methods developed while recovering a large Lightroom Classic catalog and its Lightroom cloud state. It is organized by the problem you have, with prerequisites, verification, and an exit path for each recipe. Start with the repository README for the available commands and supported configuration.

The reference environment was macOS on Apple silicon, Lightroom Classic 15.6, and Lightroom desktop 9.6. Application internals, database schemas, and native request contracts are version dependent. The public

package contains our own configurable helpers; a successful private control does not qualify a different app build, catalog, or newly generated installer. In particular, the public affinity correction and durable schema index are disabled by default pending an actual current native qualification.

What has actually been repaired

The operator reported years of slow or failed local imports, Smart Preview delays, missing originals and recurring Sync work. After the repairs below, an October card imported through Classic's ordinary UI with fast preview building, folder hierarchy and collection creation; Sync resumed roughly half a minute after collection creation. That is an operator observation, not a controlled throughput benchmark or a dated introduction point for every defect.

Use this evidence table to decide which result matches your problem. Counts refer to their exact retained scopes; overlapping phases must not be added together.

Incident or control	Actual verified result	Precise limit
Missing existing originals with exact native-cloud source bytes	14,063 original location repairs in the initial exact-SHA cohort, retaining existing records, edits and proxies	Location repair; other variants and absent originals required separate admission
Missing 2026 imports and previews	4,691 previously missing originals imported; 6,760 still Smart Previews, including 6,105 created and 655 already existing, with native availability and nonempty physical files	The initial whole scope was not a full pixel decode of every preview
A native file update scanning a huge metadata table	Existing index recovered after numeric-key/equality/plan gates; twenty native relocation bodies totaled 0.02051 seconds	Source admission and verification excluded; no whole-Sync benchmark
Downloads stranded during destination-folder resolution	2,526 folders registered by the native writer; all 3,144 planned folders verified; twenty original controls subsequently reached binary completion	Does not certify all downloads or server completion
Offline converter-built preview reuse	392 fresh preview outputs admitted and reported existing by normal SDK preview building, zero failures	A private version-specific cache route, with narrower structural/decode qualification
Fingerprints already computed but missing from the catalog	143,046 native registrations in 43 bounded transactions, independently audited without decoding originals again	Registration is not RAW-conversion pixel equality or permission to merge records
Final ten proxy-missing uploads	Ten normal retry revisions gained server proxy links and Main associations; selected edits and other documents preserved	Eight became virtual copies and two base records; missing full-resolution originals were not all recovered
Modern Cloud preview reuse	Two offline renders and Auto edits reported; 170 album proxies later natively recognized	JPEG Undo restoration was not established; all 170 were not independently edited offline
Large preview-cache publication	473,531 asset bindings across 471,373 physical previews published and audited	Cache publication is separate from settings adoption and offline editing
Legacy controls	Two legacy PENTAX photos rendered offline at Fit/100% with original-source denial	Both blocked editing until Legacy Settings Update; no update/reset/edit/export ran
Exact cached settings	One normally adopted exact XMP, unchanged develop body/revision, Waiting for Settings cleared after cold restart	Zero completed bulk adoption; the 50,443-asset roster is an admission roster

Which parts are supplied as code

The README describes the current executable package. It contains our own installation/verification CLI, native performance adapters, synthetic controls, codecs and lifecycle helper under the MIT license. It imports necessary implementation modules from the user's installed Lightroom; those Adobe modules are not redistributed.

Recipe	Public implementation or reference
Bounded map/count reuse, isolated C copying, point tracking, paired map transport, date equivalence and startup renewal	Configurable source in templates/runtime and scripts/manage.py; fresh manual native qualification required
SQL-affinity correction and `UPPER(sha256)` expression index	Optional advanced routes in the package; **both disabled by default**
Destination-folder admission	An authored example adapter, requiring independent preservation admission; it is not automatically dispatched
Location repair, import, normal preview creation, collections and ordinary retries	Supported UI/native workflows plus the recipes below; no universal bulk mutator included
Converter cache publication, format-specific fingerprints, Cloud APFS preview placement and exact-XMP adoption	Documented verified research and manual/native procedures; the starter does not install a general cache publisher or bulk settings bridge
Typed preservation and completed-action verification recovery	Reusable methodology and a held-stream example; retain local incident-specific receipts
Backup proof before deletion, damaged-original replacement and offline browsing	Owned by the sibling phindex, DNG Compare and diskproxy books; the recipes below link them and add only the Lightroom side

Source/syntax/temporary-installation tests qualify the public packaging boundary. The accepted private native controls qualify the reference catalog and builds. Neither substitutes for the actual callback, equality and restoration receipts from a new installation.

Where it fits

This book is one of nine on the FirstPair photo tools. Each owns one part, and The FirstPair Photo Stack shows how they fit together.

- **tiffgraph** walks a TIFF or DNG file's structure under a fixed grammar and finds its editable XMP packet.
- **phidentify** computes a photo's identities: SHA-256, DNG raw blocks, image data, block marks, Lightroom fingerprint, metadata record and preserved digest, and the skeleton that rebuilds a file from an edited copy.
- **phindex** keeps those identities and a skeleton for every file of a volume, aligns two volumes by content, and restores a file as it was before a metadata edit.
- **DNG Compare** compares pairs and preserves one side byte for byte as deltas against the other, restorable even after the other side is edited, with a searchable metadata section, through `dngc`.
- **Semantic Unison** synchronizes photo trees, sending metadata edits without resending image bytes, keeps the versions it replaces as small skeletons, and preserves what a shuttle disk brings into the same archives.

- **packdir** keeps directories of small files as SQLite archives that still work like plain files.
- **Tursolite** is the SQLite and Turso layer under packdir, DNG Compare and Mechanic.
- **diskproxy** serves a folder as a virtual disk, such as previews with camera metadata spliced in, and captures ratings into sidecars.
- **Lightroom Repair** repairs Lightroom Classic catalogs, previews and Sync without forging their state.

Stage and qualify the public installer

Run commands from the repository root and use an absolute path to your intended catalog. The README is authoritative for current CLI requirements. A reviewable first step is:

```
uv run --no-project python scripts/manage.py install \
  --catalog /absolute/Example.lrcat \
  --backup-confirmed --stage-only --dry-run
```

`--backup-confirmed` records your explicit confirmation that a suitable backup exists; it does not create or verify one. Choose a private installation destination with `--root` when needed. Removing only `--dry-run` creates an inspection fixture. A stage-only fixture cannot dispatch or enable automation.

After reviewing the generated source and manifest, quit Classic normally and make an actual installation at a fresh owned destination with neither `--stage-only` nor `--dry-run`. Then relaunch the intended catalog and invoke its installed native menu manually for qualification. Use the uv-managed interpreter tested by the README; the installation pins the actual interpreter and helper dependencies it uses.

Default staging leaves affinity changes and the durable schema index off. `--enable-affinity` and `--enable-schema-index` are explicit advanced choices with fresh native admission gates, not a way to inherit the private catalog's old acceptance. Bounded map/copy/date wrappers also require actual native qualification after manual entry.

After the genuine native terminal, inspect the receipts with:

```
uv run --no-project python scripts/manage.py verify \
  --root /absolute/private-install
```

After the required actual seven-phase callback/receipt qualification, quit Classic normally before enabling startup automation:

```
uv run --no-project python scripts/manage.py enable \
  --root /absolute/private-install --accept-menu-automation
```

Stop future checks with `scripts/manage.py disable --root /absolute/private-install`, using the same runner. Disable stops automation; follow the README's native restoration rules for any current owned runtime guard. Do not cancel pending requests, force a completion flag, or treat staging/static compilation as a live acceptance.

Choose the problem before choosing the repair

What you observe	First useful question	Recipe
A missing-photo badge after moving a drive	Does the original still exist, and is it the same source?	Repair locations
Sync progress appears stuck	Which exact stage is making, receiving, or completing work?	Separate Sync stages
A selected upload has a proxy-missing error	Are the expected historical proxy and current source the same admitted identity?	Recover a proxy upload
Dates gain `.000` and metadata work returns	Is the native assignment provably redundant, and is queue removal durable?	Date requeue
Each metadata save or relocation takes seconds	Is native SQL scanning a large table despite an existing index?	Diagnose SQL affinity
CPU remains high with tiny Sync changes	Is a full mapping table repeatedly queried, copied, or transported?	Reduce map work
Downloads repeat but do not become catalog items	Does the destination resolver need a catalog folder on a reader?	Repair folder admission
A preview exists but editing is blocked	Are image data and the exact requested settings both available?	Recover settings
An import succeeded but its verifier failed	Did the native action finish before the diagnostic error?	Recover verification
Several files look like duplicates	Which identity has actually been proved?	Classify identity
A converter preview or shared cache seems reusable	Do current native identities and actual adoption agree?	Converter reuse, Cloud clones
A metadata badge conflicts with expected edits	Which catalog, sidecar or embedded fields differ?	Metadata divergence
You want to delete originals that a backup should already hold	Is every source file accounted for by content, and can it be restored byte for byte?	Prove a backup
A photo will not render or edit, though its thumbnail looks fine	Is its raw data truncated, and does an intact conversion of the same exposure exist?	Replace a damaged original
The originals are offline and you want to cull in another application	Can previews carry camera metadata, and where do the ratings go?	Browse offline

Establish a recovery boundary

Prerequisites. Identify the active catalog in Lightroom's UI, the exact application version, and the physical locations of originals and previews. Decide what may change and what must remain equal. For a repair, that

usually includes existing image/file IDs, virtual copies, develop settings, history, collection membership, and cloud associations.

Procedure. Make a normal catalog backup and a separate backup of originals. Adobe's catalog backup does not include the photographs. When an advanced repair affects companion stores, keep a quiescent recovery copy of the relevant catalog family, catalog data, Sync/helper stores, and preview families as well. Quit normally and establish process absence before treating a raw filesystem copy as a closed snapshot. Adobe: back up a catalog.

Record the selected scope before acting. A useful repair manifest binds an existing catalog identifier to its current literal path, source identity, intended destination, and the reason the source is accepted. Keep source and destination copies while the repair is under verification. Use a unique run directory and retain failure receipts.

Verify. Reopen the recovery catalog when practical, check the intended photos and their edits, and independently verify copied files. A successful copy exit code alone is not a byte comparison. For database work, compare logical records and physical-family identities separately: a consistent read can be valid even when WAL or SHM bookkeeping changes.

Undo. Restore the relevant closed family together, or reverse a verified location change through Lightroom. Preserve work made after the baseline before restoring it. A local rollback cannot recall cloud activity already accepted by a server.

Limits. An APFS clone on the same disk is useful rollback material, but is not an independent disk backup. A baseline hash cannot prove that later files are unchanged without a current check.

Get an observation that answers one question

Prerequisites. Name the question and exact scope before opening a database. Decide whether persisted catalog state is enough, or whether you need the application's live owner and callback.

Procedure. Prefer normal UI and documented SDK observations. For advanced persisted-state inspection, use a read-only SQLite connection, query-only mode, disabled trusted schema, and one consistent transaction. Bind the exact canonical database path and current app process where the question requires a live catalog. Capture before/after identities of Main, WAL, and SHM. Inspect `EXPLAIN QUERY PLAN` for joins and repeated lookups before running them at scale.

A native observation must run in the correct owner. Lightroom's SDK/UI Lua state and native Main server are separate states; a UI global lookup cannot establish that a Main-owned request or retained snapshot is missing. SourceSync also has its own owner. Use the owner that possesses the object, rather than initializing an unrelated service to look for it.

Verify. Retain typed values, including absent keys, nil, false, true, and storage types. Record the actual callback or error and close only the owned diagnostic handle at its genuine terminal. Do not infer an operation's completion from a missing callback or a callback registration method that does not replay old completions.

Undo. Close the read transaction and diagnostic resources. There should be no catalog state to undo.

Limits. Persisted SQLite rows are not an Adobe-server acknowledgement. A read-only transaction is a logical snapshot, not proof that the physical database family remained unchanged. Never write Lightroom status, revision, queue, availability, or completed flags through an external database connection.

A read-only check changes the closed catalog's sidecars

Symptom. A later closed-catalog check refuses a new WAL/SHM family even though the Main file identity is unchanged. A read-only SQLite connection can still participate in WAL bookkeeping. Checking only the preparation reader can miss readers inside worker preflight, current-row validation or alias queries.

Prerequisites. Establish actual normal Classic closure, settle its helpers, and retain the operation's process/descriptor observations and resource owner. Bind the Main identity, companion state, permitted jobs and a finite reader lifetime. An absent process or an empty WAL alone does not establish this boundary.

Procedure. In a qualified closed-catalog workflow, route every Main query through the same bounded immutable reader contract. Keep indexed query-plan, row-cardinality, argument/job, before/after identity and close-ownership checks. Use `mode=ro&immutable=1` only when that contract establishes a stable database and accounts for its WAL state. SQLite documents that immutable mode omits locking and change detection; a changing file can yield incorrect results. SQLite URI parameters.

Verify. Exercise the actual emitted worker reader expressions against a disposable database with the production job parser. Verify that no sidecars appear and that stale gates and substituted jobs refuse before querying. Then verify a real publication followed by its next closed-epoch continuation. Source tests alone do not qualify that native sequence.

Undo. Preserve the original refusal and successful earlier publications. Do not remove WAL/SHM files or rewrite the expected state to make a check pass. Resolve uncertain owners first; obtain a fresh normal lifecycle boundary when needed and issue a distinct request. Do not repeat a pending native operation.

Limits. This is not an immutable-read recipe for an open or changing catalog. It does not authorize ignoring committed WAL frames, externally writing Main, or treating a known read-only sidecar change as proof of broader integrity.

Verify a completed receiver without replaying its initial transfer

Prerequisites. Retain the accepted native transfer result, its source and destination scope, original direction policy, adopted owner record and declared preview aliases. Distinguish that historical acceptance from current state.

Mechanism. An initial receiver may force one source toward its destination. After takeover, the destination can contain newer catalog edits and newly built previews. Repeating the forced profile is a write operation, not a coherence check. Native discovery-cache reuse does not make that direction safe.

Verify. Authenticate the saved acceptance and owner-adoption controls. At an authorized owned boundary, compare the current owner, exact alias targets and expected volume/directory identities, then verify the current catalog launch, application version and required startup phases. Keep later generated previews and any subsequent synchronization direction in a separate reviewed scope. Reusing the saved native assurance does not prove current whole-tree equality.

Undo and limits. This verification does not change data or transfer authority. On a mismatch, retain evidence and stop before synchronization. Do not rerun an old forced profile, overwrite a newer catalog, or rehash/recopy the whole cache merely to repeat an already accepted receiver. A changed source or conflicting scope needs a new direction and preservation review. Server convergence remains a separate native observation.

Repair locations without reimporting

Prerequisites. The original catalog entry exists. The proposed destination source is accepted under a stated assurance, and filename/capture/camera evidence does not conflict. Bind the exact current IDs, full folder membership where applicable, destination collisions, aliases, and existing edits before writing.

Procedure. For ordinary use, select **Locate** from the missing-photo badge, or **Find Missing Folder** for a missing folder. These restore the catalog link to an existing source. Adobe documents both routes. Adobe: locate missing photos.

For a controlled batch, use Lightroom's normal file/folder services with bounded transactions. A whole verified leaf folder can be much cheaper than a series of file operations, but only when full membership is known and native folder merging is not required. Small individual batches remain appropriate for mixed destinations or ambiguous names. Do not use Add to repair an existing location.

Verify. Compare every intended path and existing ID after normal completion. Compare develop settings, all history, virtual-copy membership, and relevant cloud associations. Check that source bytes and physical identities stayed within the accepted assurance. Name expected native counters or path fields explicitly rather than calling every field unchanged.

Undo. Keep the prior mapping manifest. Relink back through Lightroom if the old accepted source remains available, or restore a closed recovery copy after protecting later edits.

Limits. In one later completed 121-file control, the existing IDs/settings and 49 Main cloud associations were preserved; no files were imported or bytes moved. The assurance was the operator's verbatim-copy assurance plus exposure correspondence. It was location repair, not a new whole-file SHA, full-resolution pixel, or cloud-original identity proof. Its retained completion closes that workset; a new repair requires its own scope.

Find an actually missing lookup index

Prerequisites. Capture the exact predicate and current schema from a consistent observation. Confirm that the lookup is repeated, that its plan scans, and that no equivalent index already covers it. Preserve a verified closed catalog family before a durable schema operation.

Procedure. Two distinct findings need different remedies. The first asset/catalog lookup scanned 701,086 association rows. A nonunique `(ozAssetId, ozCatalogId)` index changed that two-key equality lookup to `SEARCH`. The initial closed-catalog operation took 0.496 seconds, preserving row counts and probe answers with integrity `ok`. The later callback went from 904.658 seconds in one earlier run to 0.723 seconds after restart, with different concurrent activity. It is an observed practical improvement, not a controlled universal speed ratio. The public starter does not reproduce that original external closed-catalog operation.

The second predicate was `UPPER(sha256)`. An ordinary index on the untransformed value did not cover that exact expression. A schema-only expression index changed its plan. A first closed-catalog candidate disappeared on reopen; source review identified table-scoped undeclared-index cleanup consistent with the observation, but did not trace the exact live deletion. Repeating a prelaunch change would not establish persistence.

The accepted successor creates the exact owned `UPPER(sha256)` index through the native writer after schema initialization. Its native transaction checks catalog/owner, existing index ownership, row counts, `total_changes`, integrity and the final plan. One native control took 2.024 seconds for 316,566 metadata rows without changing data rows. The public `--enable-schema-index` route implements this separate opt-in mechanism; it needs a verified backup and current native acceptance.

Verify. Retain exact index SQL/name, selected-answer equality, unchanged data-row hashes or a justified complete equivalent, target integrity, native commit/autocommit outcome, and the actual indexed plan. Check again after an ordinary restart. A created file or successful menu entry is not proof an index survived initialization.

Undo. A durable index survives normal quit. Use a reviewed native owner-bound removal of the exact owned index, or the verified closed-family rollback with later work protected. The starter never drops a foreign index to make ownership pass and does not remove catalog indexes externally.

Limits. Index existence is insufficient when the predicate's expression, affinity or collation differs. Adding a covering full-map index in an isolated copy improved one full query only modestly; it cannot remove repeated full-map construction. Diagnose that separately.

Diagnose SQL affinity before adding an index

Prerequisites. Obtain the exact query or native TEMP-trigger SQL from the correct live owner. Preserve its original definition. Establish a recovery boundary and a small legitimate current control scope.

Procedure. Check column affinity and the actual plan, not merely whether an index exists. A column with no declared affinity compared to an integer-affinity expression can change how SQLite plans the lookup. In the reference catalog, a file-update trigger repeatedly searched roughly 51 million fine-grained Sync metadata rows even though an image/payload-key index already existed.

A synthetic example captures the issue without reproducing Lightroom's code:

```
-- Inspect these on a scratch database with known numeric keys.
EXPLAIN QUERY PLAN
SELECT * FROM events WHERE image IN (SELECT id FROM pictures);

EXPLAIN QUERY PLAN
SELECT * FROM events WHERE image IN (SELECT +id FROM pictures);
```

Here `pictures.id` is an integer primary key, while `events.image` has no declared affinity. Unary + removes the expression's affinity, as described by SQLite's expression-affinity rules. It is a candidate correction only after proving selected-row equivalence. Numeric text is a counterexample to blanket substitution, so a global key gate must admit only numeric integral values, including integral SQLite REAL values, and reject incompatible storage types.

The public helper's advanced affinity route therefore starts disabled. Its admission must freshly bind the original TEMP SQL and current Main owner, check the complete global key population, dynamically select twenty legitimate current root-file controls, compare every old/new selected row for those controls, and require the corrected indexed plan. It changes only the qualified expression and preserves native counters and deletion semantics. It does not patch the signed application or add a redundant image index.

Verify. A scratch benchmark qualifies the expression, not the live repair. Require an actual native callback, final trigger ownership, and independent path/edit/source comparisons. The actual 20-file native relocation pilot measured 0.02051 seconds total, with a 0.00088-second median API body after the guard; that excludes preflight, queueing, hashing, and external verification. It is not a whole-Sync speedup.

Undo. Restore the exact owned original TEMP definition using the native owner. If ownership changed, stop instead of overwriting someone else's definition. Per-operation controls restore before commit; separately authorized bounded live guards have their own restoration and renewal receipts. A normal quit removes runtime TEMP state, but do not interrupt a pending transaction merely to remove a guard.

Limits. External connections cannot see Main's connection-local TEMP triggers. An external diagnostic can suffer the same affinity problem without proving a Lightroom regression. A cap overflow is also a diagnostic failure: log the query, selected row count, cap, and plan before adjusting only the justified cap.

Reduce large-map work without sharing mutable state

Prerequisites. Bind the exact map-producing query, function, request, and consumer. Measure querying, copying, transport, and callback work separately. A native stack sample is useful evidence, but does not by itself identify a Lua caller.

Procedure. The reference Sync maps had over 700,000 entries. The exact pending-map finding was narrower and more revealing than "large catalog": a lookup rebuilt the full current-catalog map when its cache was absent, the requested ID was absent/falsy, or its value was `pending`. A reload could leave that same value pending, so the next lookup did the same work.

The ordinary successful upload-preparation route asks for tracking in both asset preparation and develop preparation. In the inspected 695,483-entry snapshot, an image that remained pending could therefore cause two full rebuilds, about 1.39 million materialized rows. Assigned images, early returns and intervening pending-to-assigned transitions do not necessarily take that path. The source establishes the route; exact SQL invocation counts inside every timed request were not dynamically traced.

Forty-nine observed second-lookup segments had a 0.939-second median, while 258 broader import-settings requests had a 22.17-second median including their internal work and waits. Library feedback also reached the map producer: 403 native grid-info events consumed 82.194 seconds of one 82.236-second span. A clean restart into an empty source, with the expensive source collapsed and Sync intentionally paused, allowed import in a finite workaround; the combination does not isolate which UI setting caused the improvement.

Tiny changes could also cause defensive copies and complete transfers. The accepted methods address different costs:

Cost	Qualified method	Correctness retained
Repeated full-map query/count	Connection-local TEMP revision journal and incremental refresh	Native full-load fallback, group/schema/data-version checks, NULL-valued mapping eligibility
Defensive Lua copying	Native C `table.shallowcopy` after complete equality and mutation-isolation controls	A fresh separate output table, private cached source, ordinary fallback
A tracked-ID question building a whole map	Exact indexed point lookup	Current catalog and file-format semantics, regular native oracle comparisons
Full-map transport for small changes	Paired, versioned full/delta sender and receiver	Initial full state, add/change/remove, generation/sequence checks, ACK/drain and full resync

The deployed map/count guard is bounded to 7,200 seconds, 30,000 calls, and a 50,000-row diagnostic journal cap, with first-five and regular native oracle comparisons. Its C-copy phase uses three full equality/isolation controls and a benefit gate; it can skip installation when the measured benefit is insufficient. The point lookup is separately bounded and retains native comparison/fallback behavior. These are private

control bounds, not a promise about the defaults of every future release; check the configured public installer.

Verify. Modify a returned test map and prove the cached source is unchanged. Compare complete maps and counts against the native implementation, including special/NULL eligibility. Exercise removals, schema/generation changes, gaps, and fallback. For transport, require both senders restored, all envelopes accounted for, receivers drained/restored, and owned recovery state released before renewal.

Undo. Stop future owned cycles, wait for the captured native sender and receiver to finish normally, then restore exact owned methods. Never reset a pending owner to accelerate a legitimate wait. A zero-event cycle can use its full sender bound before normal cleanup; missing events do not authorize a fabricated ACK.

Limits. Sharing the mutable cached table would break caller isolation. Delta messages sent to an unchanged full-map consumer would break its contract. An observed copy-component improvement, including the roughly 2.16× benchmark in one later control, is not an end-to-end Sync or import claim. Remaining copies, serialization, rendering, and other native work still matter.

Drain and roll back a map transport normally

Prerequisites. A paired sender/receiver must own the exact native listener and its UI callback, with a full-map baseline and versioned epoch/sequence contract. Keep both Lua states separate. The normal native full-map getter and delivery callback must remain available for recovery.

Procedure. A delta receiver rebuilds its private map and gives the ordinary consumer a fresh isolated full result. It acknowledges a sequence only after the normal callback returns. Missing/gapped state asks the native owner for a genuine full resynchronization; it does not fabricate an answer or continue decoding against an unknown baseline.

Stopping future cycles prevents another controller dispatch. It does not erase the current listener, cancel a native request or restore the decoder while messages are still in flight. The sender restores only its owned methods, records its final sent count, and releases its recovery state after all real acknowledgements are present. The receiver remains available until its received count equals that final count, then restores its exact prior callback. Renewal requires both directions released in UI and Main, plus ordinary autocommit/request gates.

Verify. For nonzero cycles, require exact sequence coverage, full/delta validation, real callback returns and acknowledgements, both sender/receiver restoration receipts, and no retained owned handle. For a zero-event cycle, there should be no envelope or acknowledgement artifacts: literal final zero, sent/received zero, and explicit release of owned methods/recovery state are the proof. A native sender can legitimately reach its 2,400-second bound before that zero-event cleanup.

The retained restart control did exactly that: both zero-event senders and receivers restored, native recovery states released with no envelopes, controller `DRAINED` preceded the next maintenance dispatch. Another current WAIT resolved normally without cancellation. These prove observed cycles; they do not guarantee indefinite cleanup for every future failure.

Undo. Disable future renewal, then let the captured cycle drain normally. If a sequence error requires recovery, retain the original error and accept only the actual native full-resync and closure. Restore methods only while exact ownership still holds. If safe normal application shutdown is required, retain the incomplete cycle rather than label it drained.

Limits. Never manufacture ACK files, reset owners, clear a handle or mark completion to make a renewal gate pass. Transport recovery restores native delivery semantics; it does not change catalog/cloud queue states or establish server convergence.

Prevent format-only capture-date requeue

Prerequisites. Establish that the native setter's proposed date represents the same initialized whole-second value. Preserve capture time, original capture time, EXIF date fields, offset text, metadata, settings/history and relevant queue state. A badge or equal last-synced counter is not sufficient.

Procedure. The examined date setter could reassign already equivalent fields while normalizing whole-second text to `.000`, touching metadata and generating further work. The bounded guard skips only when its predicate proves the initialized assignment redundant. It leaves the native setter in charge of genuine changes and every unsupported/uncertain case.

The first guard admitted only a narrow no-offset class. Later new entries exposed unchanged `-08:00` date strings gaining `.000`; an offset-preserving successor was separately qualified. Detached native controls covered matching `-08:00`, `+05:30`, `+14:00` and `-00:00` offsets. Genuine one-second changes, nonzero fractions, missing initialization and mismatched offsets take the native path. A `Z` control showed different native normalization, so `Z` and offset-suffixed capture-time forms remain excluded. Thirty-two live skip records were observed immediately after installation; the capped log did not identify each offset class.

Verify. Compare native versus guarded outcomes for both redundant assignments and required changes before serving calls. Require an actual native installation callback, recorded live decisions, exact restoration ownership and a normal restart/renewal receipt. The installer itself rewrites no catalog date and withdraws no queue row. Continuing new metadata entries cannot all be attributed to that setter without captured call provenance.

Undo. Disable future renewal and let the bounded wrapper restore its exact native function, or quit normally after pending requests close. Removing the wrapper does not undo any genuine native date update already completed.

Limits. Timezones, fractional dates and initialized fields are part of semantics. Do not normalize them broadly. A finite live skip control establishes this admitted class; it is not proof all recurring metadata work is eliminated.

Withdraw proved redundant metadata work through its owner

Prerequisites. This is separate native research, outside the public installer's automatic scope. Bind exact queue rows to complete current image/file/edit/history/metadata/association facts and the unchanged managed cloud payload/revision. Preserve a closed recovery family. Prove formatting-only divergence rather than infer it from counters.

Procedure. In the reference recovery, 122,216 selected entries were withdrawn through the normal SourceSync store operation after complete per-entry admission. Nothing forged an acknowledgement, change counter, completed flag or server reply. The withdrawal changed queue membership alone.

Durability belongs to the SourceSync owner's outer savepoint and heartbeat. A 5,000-row callback completed, but a competing external live reader coincided with a native savepoint-release error; the rows all returned after normal closure. That callback was preserved as a failed-durability result. A uniquely owned successor used closed Source snapshots plus native owner observations and preserved the original failure.

Verify. Require the actual owner callback and later closed-disk absence of every targeted row, full protected-field equality and integrity. The accepted finite scope retained all 122,216 removals after normal close/restart, with protected catalog fields unchanged. A native dirty flag or in-memory absence is not durable proof.

Undo. Use the retained genuine queue/baseline chain and normal owner procedures, or the appropriately scoped closed-family recovery with later work protected. Do not repeat the held original dispatch or manufacture an earlier successful receipt.

Limits. `lastSyncedChangeCounter` has both monotonic and "assign current" native routes. Its equality is not independent server-upload acknowledgement. Residual entries must not be withdrawn on that test alone, and a format-only queue withdrawal does not prove full cloud convergence.

Repair folder admission for download workers

Prerequisites. Identify the effective native download root and the exact queued assets. Preserve their pending state and ordinary collision/SHA rules. Distinguish the physical directory from its catalog representation.

Procedure. In the reference failure, the full native destination resolver attempted to create/register a missing date folder while running through a read-only owner. A reader-side exception could leave worker slots occupied. The finite remedy first planned paths with the resolver's path-only mode, then created the necessary catalog folders through the normal native writer in bounded transactions. Ordinary workers subsequently used the unchanged full resolver.

Verify. Require actual reader-resolution callbacks after folder admission, not just directory existence. Check ordinary worker transitions and the source/destination identity gates. The retained control first verified six folders, then registered 2,526 missing catalog folders in 26 bounded native transactions and verified the entire 3,144-folder roster across 63,101 queued assets. After a normal quiescent restart, all twenty original controls completed their binary stage. Those scopes do not prove all downloads or current server convergence.

Undo. Folder registration and normal worker effects are catalog changes. Keep the before-state and recovery copy. Use normal folder management or rollback where appropriate; never delete folders containing accepted images as a shortcut.

Limits. A pending path can be recalculated by the normal resolver. An old queued path alone does not prove the next download will use it. This folder-admission remedy is distinct from the file/folder-rename SQL-affinity guard; it is not implied by installing that guard. Do not rewrite pending paths, force processing flags, or use a read-only menu to submit a disguised mutation.

Renew bounded fixes after launch

Prerequisites. The operator explicitly enables startup automation and guarded renewal. Pin the app version, installer, dependencies, and helper; grant macOS Accessibility/Automation access to the exact helper if requested.

Procedure. The private installed helper watches the exact Classic executable's PID and launch time. A LaunchAgent invokes its check every thirty seconds; it does not watch photo folders or import files. After the correct catalog is available, the helper invokes the normal performance menu once for that process lifecycle. A durable intent prevents another click when dispatch or completion is ambiguous.

The public package dispatches its exact installed **Apply FirstPair performance fixes** menu. It checks the menu, catalog, process lifecycle, nonce and pinned dependencies rather than assuming a position in the Scripts menu.

Inside Lightroom, a five-minute maintenance check renews separately bounded guards only when the current owner, prior native request, and transport-drain gates permit it. It stops on a wrong catalog or native failure. Current private phase numbering is 0–6: SHA expression index, incremental map/count, isolated C copy, indexed tracking lookup, modification affinity, capture-date equivalence, and folder/file-rename affinity. The paired map transport starts only after the normal installer terminal.

Verify. LaunchAgent installation is not proof of native installation. Require the actual UI entry, genuine native callback, every required phase receipt, and separate transport startup outcome. Test a normal fresh-process restart and an actual later renewal. Check serving/oracle observations separately from initial installation. The retained private controls include successful normal restarts and guarded renewals, including all seven phases.

Undo. Disable future launch checks and renewal through the published uninstall instructions, retain their receipts, and let current owned native requests finish. Allow normal bounded restoration or a normal quit once safe. Remove only files owned by this installation. Do not force-kill Lightroom or cancel requests as an uninstall method.

Limits. Some fixes are runtime-only; the SHA expression index is a schema change. Reapplication after launch is not an object surviving quit, an indefinite lifetime guarantee, or a benchmark of sustained throughput. An app update or changed dependency must fail the pin gate and require a fresh qualification.

Separate Sync stages and counts

Prerequisites. Define which photos, collections, revisions, and direction you are measuring. Keep UI counts, pending catalog mappings, downloader jobs, binary completion, and server evidence as separate populations.

Procedure. Use ordinary collection Sync activation. Adobe documents that Classic photos sync through synced collections or All Synced Photographs, and that Classic-to-cloud sends Smart Previews while cloud-to-Classic downloads originals. Adobe: Sync setup, Adobe: Sync FAQ.

For a batch, retain exact collection names, parents, and full existing-photo membership before enabling Sync. Record all five completion arguments when the installed command contract is `(self, context, descriptor, success, error)`. A returned command-spec is not a completion promise, and registering a listener after the event may not replay it.

Verify. Compare complete final collection IDs, names, parents and member IDs, then separate album mapping, still mapping, binary transfer and server acceptance. The retained batch checked 69 collections and 19,314 memberships; all album mappings became concrete while some still-image mappings remained pending. Its 283 videos belonged to membership totals, not the still mapping denominator. This is finite native mapping evidence, not an independent Adobe-server binary acknowledgement or a global zero backlog.

Undo. Change Sync selection through Lightroom after considering its normal cloud effects. Keep the exact membership manifest; do not delete and recreate catalog records to make a counter look right.

Limits. Do not replay the completed 68-new/one-preserved collection batch. A changing UI backlog can include different queue stages; subtracting two unrelated counters cannot identify completed uploads. A

retry receipt must bind actual emitted revisions, because the native sequence is globally allocated rather than a per-document old-plus-one edit count.

Verify a Smart Preview at the level you need

Prerequisites. Bind the exact existing image/file UUID and the literal native preview path. Collision suffixes can be legitimate; do not derive every path from a UUID formula. Preserve the preview and settings baseline.

Procedure. Use normal **Build Smart Previews** for admitted sources. Adobe describes Classic's Smart Preview status below the Histogram and the ability to work with originals disconnected. Adobe: Smart Previews.

Then distinguish four checks: a nonempty file; a structurally decodable preview; a native application render; and editing with the required settings. A full tile decode is stronger than a header check, but does not demonstrate the application's offline setting/profile behavior.

Verify. Bind decode results to current catalog IDs, native status, stable preview bytes and identities, and full decode scope. For offline render testing, use an explicit network and original-source denial boundary, observe the selected exact image at Fit and 100%, and compare settings afterward. Open read-access descriptors support a file-use observation; they are not kernel read traces.

The retained reuse publication covers 473,531 asset bindings across 471,373 physical previews. Two PENTAX K-7 legacy controls actually rendered offline at Fit and 100%. Both still requested Legacy Settings Update before editing; no Update, Reset, edit, or export was completed. These controls do not establish readiness of the whole preview cohort.

Undo. Retain the originals and prior preview state. Remove only an exact owned newly published cache entry when it is proven safe and no native operation still owns it; avoid deleting the whole preview family during recovery.

Limits. A preview cannot replace a missing full-resolution original for every purpose. Two legacy render controls do not establish modern external-XMP readiness or current Classic offline editing for the whole library.

Isolate a stalled Smart Preview batch

Prerequisites. Preserve the exact requested photo IDs, batch intent, returned callbacks and existing previews. Keep ownership of the native task until it has actually stopped. A responsive window or low CPU use alone does not establish completion.

Procedure. Cancel the progress task normally. If its callback remains pending, quit Classic normally, handle its task confirmation explicitly, and reopen only after native closure and resource reconciliation. Do not delete preview caches or replay the pending batch. Reconcile valid registrations and existing preview files before choosing controls.

Select one exact JPEG and use **Library → Previews → Build Smart Previews** independently of the SDK batch loop. When Classic asks whether to build all photos or only one, choose **Build One**. Test the first pending input separately, then a small mixed batch. Verify the selected IDs before every menu action; never accept a default **Build All** confirmation accidentally.

Verify. Distinguish recognizing an existing cache file from generating a new preview. In one Classic 15.6 recovery, single JPEG/HEIC controls recognized cached files; a subsequent five-photo control had no

preview files beforehand and produced five nonempty, validly registered previews through the Library menu. The bulk SDK callback had stalled. This demonstrates the tested UI route, without establishing the cause of the stalled callback or completion of the remaining queue.

Subsequent fresh SDK controls also returned normally: one JPEG, a six-file JPEG/HEIC batch, and a 25-file HEIC/JPEG/DNG batch. Each returned the exact requested created-ID set with zero failures; nonempty preview files, valid registrations and unchanged original-file metadata were independently checked. This supports restarting with bounded small batches, while preserving the earlier stalls as unresolved evidence. It does not establish that cached-but-unregistered files caused those stalls.

A later continuation stalled again within a 25-file call after 675 completed files. After normal recovery, its first missing input worked individually and the remaining nine missing previews built together. Ten partial cache outputs and seven older cached files were recognized through individual native selection without rebuilding; their file metadata stayed unchanged. This reconciled the affected inputs without establishing a damaged photo or the stall's cause. Preserve partial outputs, distinguish native recognition from generation, and retain the exact unresolved batch rather than retrying the whole queue. A subsequent window stalled after another 150 completed files. A single-file exported-JPEG control wrote its cache output but still did not return the SDK callback; native selection during that pending call did not resolve it. This separates file creation from callback completion. A correlated filename family may be routed to an explicit isolated queue, but must remain in the full scope; it is not evidence that the originals are corrupt. Do not strip metadata or rewrite originals to fit this hypothesis. A subsequent window excluding that filename family completed 5,000 new previews through 200 audited SDK callbacks and normal owner closure. This proves that finite continuation and its checked outputs; it does not establish the cause of the excluded-family stalls or complete the remaining queue.

A returned callback with explicit per-photo errors needs a different path from a callback that never returns. One later 250-photo callback reported 248 creations and two errors. Independent checks accepted the 248 registrations/nonempty files and unchanged originals; both errored HEICs decoded with macOS, which did not establish Adobe decoder compatibility or the cause of its errors. The controller's zero-error assertion had unnecessarily stopped the entire queue. Its successor requires an exact, disjoint partition of all requested photo IDs across created/existing/failed/error results, verifies successful outputs and every original epoch, durably isolates explicit failures without automatic retry, and waits for the parent acknowledgement before continuing. Missing IDs, incomplete accounting or unknown ownership still stop the job. Synthetic controls qualify that policy; each subsequent native window still needs its own output and closure evidence.

For a planned IO handoff, finish the in-flight callback and its independent audit before stopping. A stop marker should produce an ordinary partial-completion receipt with exact attempted, successful, failed and deferred counts, then release the retained owner after actual task and group closure. It must not turn a known operator pause into an unknown-ownership exception. The successor has synthetic controls for stopping after an acknowledgement and before the first call; that source qualification does not replace the next native handoff's completion evidence.

Undo and limits. Preserve generated previews and originals. Restore temporary diagnostic menu scripts and retain the original failed/pending receipts. Record recovery closure separately from successful SDK completion. A small successful batch does not qualify an unattended full-catalog controller or justify another large retry.

Keep a bulk preview build cooperative

Prerequisites. Verify the existing Classic executable, PID and launch time; record that identity and its observed scheduler state. Keep the SDK owner and normal resource/closure protocol. A token reservation does not limit Adobe's internal worker threads.

Procedure. On macOS, an operator can put that verified process into Darwin background scheduling during an unattended bulk build:

```
/usr/sbin/taskpolicy -b -p "$CLASSIC_PID"
```

The installed manual and Apple's source document this process-policy operation and its inverse. Check the command's return status and keep observing actual SDK callbacks. Apple taskpolicy manual, Apple implementation.

Verify. A disposable Python control lowered scheduler priority and restored it; applying the policy to the bound Classic process also lowered its observed priority while preview callbacks continued. These observations do not measure contention fairness or an eight-core ceiling.

Undo. After bulk work stops/completes, reverify the same process launch before returning it to normal scheduling:

```
/usr/sbin/taskpolicy -B -p "$CLASSIC_PID"
```

Record restoration when performed. A saved undo command is pending cleanup.

Limits. This changes scheduling, not core affinity, a CPU quota or an RSS cap. It does not settle native calls, release leases or authorize another build.

Reuse offline converter-built Smart Previews

Prerequisites. Originals remain retained and stable. Pin the installed converter version and explicit output options. Use a fresh scratch destination and an exact source/output manifest; never accept the converter's default destination if it could overwrite originals. The private cache layout and native preview-identity rules need separate current qualification.

Procedure. Adobe's DNG Converter is a separate batch conversion tool; its normal documented purpose is conversion of camera raw files to DNG. Adobe: DNG Converter. The reference experiment used version 17.2 to build fresh DNG 1.7, lossy JPEG XL previews with a 2,560-pixel long edge and no embedded preview. Its parallel-batch option used the observed ten-CPU queue; no numeric maximum-parallelism option was established.

Treat those outputs as proposed previews, not replacement masters. Match the exact installed preview layout, required raw-image payload and identity fields, UUID/name binding and permissions before any no-overwrite cache placement. Normal SDK **build Smart Previews** remains the adoption/control step.

Verify. A 392-file finite batch created its outputs in 198.51 seconds, around 246 MB. Guarded cache placement followed by normal SDK preview building reported all 392 as already existing in 63 seconds, with zero failures. Required hashes stayed unregistered in that experiment. Seven camera-family controls agreed on encoded native raw payload and both required identity digests; four were observed current-version creations and three older-version creations with unobserved creation steps.

The 32-missing-plus-seven-control validator initially rejected Panasonic files because it selected the wrong image directory. Its qualified successor used the declared image role to distinguish full-resolution CFA data

from auxiliary reduced linear raw data. Unknown/multiple full-resolution roles and a narrowly observed camera warning remained holds. There was no independent all-pixel decode of every output.

Undo. Retain scratch outputs, originals and the pre-publication cache manifest. Remove only proven owned, unused additions through a safe cache recovery after native requests close, or restore the protected cache family. Never replace an accepted original with a lossy preview.

Limits. This is a tested private reuse route, not a public Adobe cache-installation contract or a general batch command supplied by this starter. Successful adoption does not prove every camera, warning, profile, crop or encoded plane is interchangeable.

Map and clone Classic previews into Cloud precisely

Prerequisites. Apps are normally closed for file publication. Bind each exact Classic image/file UUID to its current Cloud asset, requested native proxy SHA, literal native destination and settings/auxiliary facts. APFS source and destination placement must support cloning. Preserve the destination's accepted prior bytes and metadata.

Procedure. Use a clone with an independent inode rather than a hard link: later writes can diverge while initial bytes remain equal. Keep a no-overwrite intent journal for new destinations. For existing cache entries, first prove their accepted whole-byte identity and preserve the old inode or a verified rollback before a separately authorized exchange. Retain the exact native path, including any collision suffix; UUID-only filename formulas can be wrong. A JPEG original can legitimately have a DNG proxy.

The full 170-photo album control had current native proxy SHA equality for every Classic preview, 116 matching existing Cloud copies and 54 uncached entries. Settings were already local; there were no local originals or pending original downloads and no server master link. That specific gate supported preview-only work; generic Make Available Offline can have a different original-download scope.

Verify. Compare every source/destination SHA, stable identities and protected filesystem metadata, then check ordinary native adoption. The initial 170 invocation published 54 new entries but held before the first existing-file exchange over an OS-added provenance attribute. Its recovery explicitly retained that exact qualified attribute, preserved all other metadata, and processed only the untouched 116. The composite proves all 170 publications without forging a successful original invocation.

A normal offline-album action did not immediately register the 54 entries. A later consistent native roster proved all 170 local proxy paths/annotations, no proxy demand, local settings and offline marking. That stronger observation supersedes the earlier hold; it does not identify the exact adopting action or prove 170 independently traced offline renders.

The larger publication independently audited 473,531 bindings. Publication, native recognition, rendered use and edit readiness remain separate acceptance steps.

Undo. Preserve prior Cloud entries and exact owned journals. Withdraw unused owned additions or reverse an authorized exchange only after native ownership and later work are accounted for. A cache clone is not a backup on another device, and adopting a file is not grounds to remove its original.

Limits. A missing expected proxy SHA needs a separately qualified identity route. The known-SHA recipe cannot silently admit that population. The two legacy tests below used such a narrower route and qualify only their tested class.

Test offline rendering and editing without overstating Undo

Prerequisites. Keep a complete current develop/history baseline and exact preview identities. Define actual network and original-source denial. For deliberate edit tests, record numeric settings before, during, after Undo and after normal reconnect; user permission for the edit is required.

Procedure. Cold-start Cloud within the defined boundary. Select the exact photo, render at Fit and 100%, and distinguish enabled controls from a settings-wait or legacy-update panel. If authorized, make one recorded edit, undo it normally, then observe both Cloud and Classic after Sync resumes.

Verify. Two modern controls, a DNG original and a JPEG original with DNG proxy, rendered while Wi-Fi was off; the operator reported Auto slider changes and Command-Z. The first persisted settings stayed unchanged. For the JPEG, a later native Classic observation received Auto settings and a new mobile-origin history step: Exposure 0⟶0.04, Contrast 0⟶5, Blacks 0⟶−14, Vibrance 0⟶19, Saturation 0⟶4. Editing succeeded, but exact restoration after Undo is **not established**. All four Classic/Cloud preview files retained their exact bytes and five-field identities even after that settings propagation.

Two later PENTAX K-7 legacy controls rendered Fit/100% under Cloud network and original-source denial with exact clone read-access descriptors. Their preview identities, Main settings/history and Cloud public develop fields stayed protected. Both demanded Legacy Settings Update before editing. No Update, Reset, edit or export was attempted. Read-access descriptors are observed open access, not kernel read traces.

Undo. Normal Undo is an operation to verify, not an automatic preservation receipt. If post-reconnect values differ, preserve the actual edit/history outcome and ask for a separately authorized correction rather than silently restoring old values. A render-only test should require no develop changes.

Limits. Two modern controls do not prove all modern external-XMP images ready. Two legacy renders do not qualify every absent-proxy-SHA asset or Classic offline editing. An early unchanged persisted snapshot can precede later normal settings Sync.

Recover exact settings without resetting edits

Prerequisites. Observe the current cloud asset/revision, exact requested XMP whole-file SHA and size, local XMP annotation/path/error state, and required masks, auxiliary data, and profiles. Keep signed links and private identifiers out of public reports.

Procedure. A preview supplies image data, not necessarily the current settings. In the examined Lightroom build, pending or failed settings status can keep Edit waiting even when a simpler download flag is clear. An exact cached XMP can be published without overwrite under its actual content hash when the current native document requests those same bytes; ordinary native selection/load must adopt it. The independent SHA/size gate is essential even if the native branch happens to accept a filename.

Do not manufacture settings from an embedded packet merely because it contains develop fields. Do not label approximate XML with the requested SHA, edit revisions or availability flags externally, or use Reset/Legacy Update as recovery of existing edits. Adobe describes ordinary catalog/XMP metadata storage; that does not document or approve this internal cache route. Adobe: metadata and XMP.

Verify. Retain full typed public develop/body fields, revisions, annotations, and separately indexed labels. Require native adoption and a post-load persisted observation. One exact standalone XMP control was adopted offline without a develop-body/revision change, and a normal same-sandbox cold restart cleared Waiting for Settings. Numeric editing was not tested. The 50,443-asset exact-cache roster is an admission roster, with **zero completed bulk adoption**. A bulk constructor attempt failed before admission; a prepared successor is not a runtime receipt.

Undo. Preserve the cache source and publication journal. Withdraw only an exact owned unused publication if needed. A settings rewrite, Reset, or new cloud revision is a different operation with different rollback consequences.

Limits. Keep 5,756 profile-info holds separate from accepted candidates. Mutable `cameraProfileInfo` metadata is neither proof of a missing binary nor permission to ignore the dictionary. Required masks/profiles and rendering fidelity remain real gates. Whole admitted-preview embedded-XMP scanning found zero exact requested-hash matches across two temporal read-only snapshots; do not call them one atomic snapshot or treat old packets as current settings.

Recover a selected proxy-missing upload

Prerequisites. Identify the exact current Cloud document/revision and error, expected proxy bytes and current local source. Keep independent settings and every other full document protected. Verify Main associations and native path-producer semantics. A global UI issue count is not a selected error inventory.

Procedure. The final twelve-item incident contained `UploadFailedMissingProxy`, not twelve healthy transfers. Two current records completed ordinary retry. Ten retained obsolete volume/catalog paths; nine old proxies matched their retained expected bytes and one required replacement. Current originals had differing metadata versions and could not simply replace historical identities.

The accepted remedy used the normal native public cache setters to admit the exact reviewed sources, then one stock **clear errors and retry Sync** request. It did not modify Cloud SQLite or fabricate link/status fields. Its setter chain included genuine completed-but-verifier-held steps; independent outcome assembly preserved those failures and resumed only unfinished setters.

Verify. Require exact full-document comparisons for selected and other records, current source/path/byte gates, genuine setter callbacks and the actual stock retry's emitted revisions. The ten gained valid revision-bound server proxy links, zero stored upload errors and one Main association each. Lightroom created eight virtual copies sharing existing files and two base associations, preserving distinct Cloud settings. Current Sync Preferences reported no activities within that finite observation.

The native `_changeCounter` is a global allocated revision sequence, not a per-photo edit count. A strict old-plus-one checker held after the actual retry; independent installed-counter and outcome verification accepted the result. No retry was repeated to satisfy that checker. Native label visibility also changed when a setter created a new body revision; that exact proven transition was accepted, without broadly ignoring labels.

Undo. A server-accepted retry cannot be undone by deleting a queue row. Preserve the current associations, independent edits and source versions, and use ordinary scoped management or recovery with later cloud effects accounted for. A verifier failure alone does not justify repeating setters or retry.

Limits. This cleared the exact ten errors; it did not recover every full-resolution original or reconcile every Cloud issue. Proxy errors explain these uploads, not every Smart Preview stall. Earlier preview work also encountered map/count delays and resolver failures; a candidate whole-catalog preview-cache invalidation route was researched without establishing it as the runtime cause of every stall.

Recover verification after a successful native action

Prerequisites. Preserve the original native journal and failed wrapper receipt. Determine whether Add, relink, preview construction, or retry already completed.

Procedure. If the action completed but the external verifier failed, correct and rerun only verification in a uniquely named recovery. Never replay Add to obtain a cleaner receipt. A concrete example was a TIFF decoder receiving an already held stream whose `.name` was an integer file descriptor. The verification fix is:

```
with tifffile.TiffFile(stream, name=str(path)) as tif:
    # Verify the intended image while continuing to use this held stream.
    ...
```

The explicit name supplies decoder metadata without reopening the pathname or weakening held-descriptor/source guards.

Verify. Bind the recovery to the original actual action and its source/preview identities. One new DNG import and Smart Preview were fully accepted through such a recovery, including all 28 tiles of its 2560×1693 preview. The original exit-one receipt remains. Two other proposed imports are still held; the successful control does not admit them.

Undo. Undo the native action only if it is actually undesired, through its normal route and with preserved later work. A diagnostic failure by itself calls for verification recovery, not rollback or repetition.

Limits. Do not clear a completed UI owner to satisfy a Main guard. Historical owner identity is provenance; validate its retained native snapshot in the proper Main-owned preflight.

Classify identity before merging or deleting

Prerequisites. Retain both versions and independent catalog/cloud records. Write down which identity is needed for the intended action.

Evidence	What it supports	What remains unproved
Filename/counter resemblance	A search candidate	Same camera, exposure, bytes, or safe duplicate
Capture time, camera/serial, complete exposure UID	Exposure provenance	RAW-conversion pixel equality or whole bytes
Current native-original fingerprint match	Native matcher correspondence	Pure pixel or whole-file equality
Stable full SHA plus size	Whole-file byte identity	Interchangeable edits, virtual copies, or cloud records
Full decode	Structural/decoder success for that file	Original provenance or another file's identity
Native rendered image under denied originals	Functional preview use in that control	Full-resolution source recovery or all-library eligibility

Procedure. Route by indexed filename/path/native fingerprint candidates, then inspect current capture/camera/serial and retained original-name evidence. A same-second import hash can belong to a different camera. A DNG or JPEG alternative may represent an exposure without being interchangeable with its source. If a bounded header shows a declared RAW strip extending past the stable file length, hold it from Add and repeated known-failing decode; search for an intact original or qualify preview-only salvage separately.

Verify. Record the consulted scope and current associations. Native fingerprint candidates that intentionally lack a peer native oracle remain candidates. An empty candidate-match array is not necessarily a

mismatch. A duplicate-looking cloud record can contain independent edits even when it shares a file with another record.

Undo. Keep source versions until preservation, current-reference, settings, association, and action-specific gates all pass. Archive differing versions under truthful content identities rather than overwriting the accepted master.

Limits. A finite search with no peer is not universal proof of absence. Filename digits are not independent camera-counter evidence. Byte equality does not authorize merging records or deleting one copy.

Route native fingerprints by format

Prerequisites. State which identity the operation requires: bytes, exposure provenance, decoded image content, native original matcher or Adobe raw-image digest. Bind exact catalog format and physical format separately; path extensions can disagree with the catalog. Keep a native oracle for each admitted format/metadata stratum.

Procedure. A full-file SHA is a byte identity. Adobe's DNG raw-image digest and Lightroom's original matcher have different inputs and purposes. A container's digest tag is not automatically the value Lightroom calculates for original matching. DNG crop, camera/opcode/orientation context and selected raw planes matter; same exposure metadata does not prove a RAW⟶DNG conversion has identical pixels or interpretation.

The JPEG route also needs its exact metadata stratum. Date/timezone/fractional/NUL/XMP handling and encoded image/ICC data cannot be generalized from one camera control. A current finite Samsung JPEG qualification matched 26 actual native source oracles; 26 peer calculations then matched those fingerprints. The peers intentionally lacked independently observed native oracles, so they remain matcher candidates until fresh native/current/metadata gates. An empty generic match list in that manifest was not a mismatch.

Other container, enhanced/multiplane, JPEG-encoded DNG and unsupported decoder strata retain separate routes/holds. A header that proves a declared raw strip extends beyond the stable physical file length establishes structural truncation without a pixel decode. Do not repeatedly submit a known-failing source; find an intact original or explicitly qualify preview-only salvage.

Verify. Compare the exact candidate engine output against actual native `calculateDigest` for admitted controls, retaining current source descriptors, format/metadata facts and genuine terminal. A later independently audited phase registered 143,046 already computed fingerprints in 43 native transactions without rereading original pixels, preserving settings/history/metadata/cloud scopes. Those registrations are a catalog fact; they do not turn every historical calculation into current registration or authorize merging variants.

For identity routing, use the actual indexed filename leading key while retaining returned literal names and exact paths. Original import-name evidence can expose arbitrary ingestion renames. One finite 178-source route found 171 represented exposure variants and seven unresolved candidates; later complete exposure UIDs distinguished consulted peers. Different UIDs close that consulted metadata route only. A same-second import hash can come from another camera, and no peer in one index is not universal absence.

Undo. Preserve historical native receipts and existing digest rows. Never overwrite an equal native digest merely to normalize file size, invent a missing oracle, or delete a separate Cloud record with independent edits. Registration reversal, if actually needed, belongs to a separately reviewed native scope or its preserved recovery family.

Limits. Native original matching is not pure pixel equality. Pixel equality is not interchangeable develop settings. Byte equality is not automatic duplicate-deletion authority. Keep conversion variants and virtual copies until an action-specific preservation decision is made.

Prove a backup holds every photo before deleting it

Symptom. A source tree, such as `/Volumes/Archive/Photos`, should already be held by a backup, such as `/Volumes/Backup/Photos`, and you want to free the source. The backup was made over years: folders moved, files were renamed, re-dated or rewritten with new metadata, so a path-by-path comparison reports thousands of differences that may or may not matter.

Prerequisites. Both trees are mounted and stable for the duration of the run; nothing imports, converts or edits into either. Install the indexer from phindex and `dngc` from DNG Compare; those books own their installation, options and archive layout. Choose an archive directory that is not on the source you mean to delete. Know which Lightroom catalog references the source, because deleting it makes those photos missing in that catalog.

Mechanism. Index both trees, so that every file has its identities: SHA-256, DNG raw blocks, image data, Lightroom's fingerprint and a metadata record.

```
phindex --db source.sqlite --root /Volumes/Archive/Photos --workers 8
phindex --db backup.sqlite --root /Volumes/Backup/Photos --workers 8
```

Then account for the source in the backup by content rather than by name. `phindex align` groups files into units (a raw with its JPEG and sidecar), matches each unit by its strongest shared identity, and flags moved, renamed and split units. Units with no counterpart are reported as `left-only`; files not yet read as `not-indexed`.

```
phindex align --left source.sqlite --left-root /Volumes/Archive/Photos \
  --right backup.sqlite --right-root /Volumes/Backup/Photos --out align-report/
```

`dngc run` turns that accounting into a preserved, restorable record. With the indexes it pairs files through `phindex align`, settles unchanged pairs without reading them, and keeps every source file in one of three forms: identical to its backup counterpart, a verified delta against that counterpart, or a full copy when the backup has none. `--metadata phindex` takes metadata from the indexes and writes a searchable metadata section.

```
dngc run /Volumes/Archive/Photos /Volumes/Backup/Photos /Volumes/Backup/source-proof \
  --index source.sqlite backup.sqlite --metadata phindex
dngc verify /Volumes/Backup/source-proof --restore-sample 50
```

Equal raw pixels are not permission to delete. Two files with the same raw image can differ in the DNG wrapper, embedded previews, XMP, ratings, keywords and develop history; the source copy may carry the only record of edits made after the backup. That is why the archive keeps a delta for every pair whose bytes differ, instead of declaring the pair equivalent and dropping the difference.

Verify. Read `align-report/summary.json` and account for every `left-only` and `not-indexed` unit: each must either be preserved as a full copy by the run or be deliberately excluded with a reason. Require `dngc verify` to exit 0; it checks the byte records, deltas and originals, the metadata index and that the comparison finished, and `--restore-sample N` restores N records at random and checks their SHA-256. While the source still exists, restore a few files of your own choosing, including a renamed one and one with a metadata difference, with `dngc restore ARCHIVE RELATIVE_PATH OUTPUT`, and compare each with the source file

byte for byte. Only then delete, and settle the source in Lightroom's terms: relink the catalog's photos to the backup through Lightroom (Repair locations) or accept that the catalog will show them as missing.

Undo. Nothing above writes to the source or the backup tree; the indexes and the archive are new files. After deletion, `dngc restore ARCHIVE RELATIVE_PATH OUTPUT` rebuilds any source file byte for byte, and `dngc versions` lists the contents a path has had.

Limits. A delta is rebuilt against its backup file, so the archive and the backup tree are one unit: keep both, keep the backup's files unedited, and keep a second copy of the archive on another device if the source was the only other copy. `--metadata phindex` records come from the exiftool-rs engine, a separate metadata identity from Perl ExifTool; never compare an archive made one way with one made the other. A random restore sample is evidence, not proof for every record. This cookbook links the owning books for their tests and qualification; it has no retained deletion control of its own, and the procedure says nothing about cloud copies, which Lightroom manages separately.

Replace a damaged original with its intact conversion

Symptom. An original DNG is truncated on disk: Lightroom cannot build a preview or reports the file as damaged, and editing fails, yet Finder, Quick Look or `sips` thumbnails look fine. Elsewhere, an earlier converter's output of the same exposure, such as an Adobe DNG Converter run kept on `/Volumes/Backup`, is intact.

Prerequisites. Detect damage at the pixel level, not from preview renders. Thumbnailing tools such as `sips` render the embedded preview, which can be complete while the raw data behind it is truncated. A full raw decode, a pixel comparison in DNG Compare, or a header showing a declared raw strip or tile running past the file's stable length establishes the damage; see Classify identity. Then prove the conversion is the same exposure (capture time, camera and serial, exposure UID) and that its own raw data decodes in full. Record the damaged file's path, exact name, SHA-256 and size, and the photo's develop settings and history as Lightroom shows them.

Mechanism. Lightroom finds an original by its path and name. Putting an intact file of the same exposure at that exact path and name lets the existing catalog record, with its edits, history, virtual copies, collections and cloud association, point at readable raw data again, without re-registration.

Quit Classic normally. Move the damaged files aside to a folder outside every catalog folder, mirroring their relative paths, and keep them. Copy the conversion to the original's path under the original's exact name and extension, and check the copy's SHA-256 against the conversion. Reopen Classic, select that folder, and rebuild its previews through **Library → Previews** (standard or 1:1, and Smart Previews where the photos had them).

Do not write the catalog, re-import the file, or use **Read Metadata from Files** to clear a metadata badge: the conversion's embedded XMP is not the catalog's current state. Resolve any badge with Reconcile catalog, XMP, ratings and dates.

Verify. In Lightroom, the folder shows no missing or damaged-file badges, and each replaced photo renders at Fit and 100% from the new original. Develop settings, history, virtual copies and collection membership match what you recorded. On disk, the placed file's SHA-256 equals the conversion's, and the damaged files remain intact in the side folder.

Undo. Quit Classic, move the conversion out, return the damaged file to its path and name, and rebuild the folder's previews. The catalog was not changed by the swap, so nothing else needs reversing.

Limits. Lightroom's registered fingerprint and any cloud original were computed from the old bytes and are not updated by the swap; duplicate detection and Sync may treat the new file differently. Develop history was made against the old file's raw data: a conversion with a different crop, opcodes, linear or mosaic data, or converter version can render the same settings differently, so inspect every replaced photo in Lightroom rather than assuming equality. This is a procedure, not a retained native control in this cookbook's evidence table.

Browse and rate with the originals offline

Symptom. The originals are on a disk that is not attached, but you want to cull them in another application, such as Photo Mechanic, which needs camera metadata and a good-sized preview to be useful.

Prerequisites. Previews already exist: Lightroom Smart Previews, or a folder of regular preview JPEGs. Install diskproxy and its previews plugin; that book owns the configuration, the map-file format and the writable modes. Choose a ratings folder outside the source. Serve previews that Lightroom is not rebuilding at the same time: a served file that changes after the disk is built fails to read.

Mechanism. diskproxy presents a folder as a virtual disk without writing to the source. The previews plugin composes files in one of two ways:

- **Smart Previews as DNG stand-ins.** Each Smart Preview is served as a DNG with the large preview JPEG added as an extra embedded preview; every original byte stays where it was. Browsers that show embedded previews see the large JPEG, and the stand-in carries the Smart Preview's camera metadata.
- **Preview JPEGs with EXIF stubs.** Each JPEG is served with camera, lens, exposure and capture metadata spliced in from a stub: an EXIF-only TIFF per photo, written from the phindex metadata records and stored in one SQLite file (`metadata_store`) rather than as hundreds of thousands of small files.

With a writable view, diskproxy accepts the application's writes into a shadow file and the plugin keeps only ratings, labels, color classes and tagged flags, as one small XMP sidecar per photo in the ratings folder, with every change appended to `JOURNAL.jsonl` together with its previous value. Nothing reaches the source.

Verify. Check one file through the mounted volume before culling, for example `exiftool -b -PreviewImage` piped to `wc -c`, which should give the size of its preview JPEG, and confirm that the application shows the camera and capture time. After rating a few photos, confirm that their sidecars and journal lines exist after `diskproxy stop`, and that the source tree's files are unchanged.

Undo. `diskproxy stop` unmounts the view and deletes the session's written bytes after the final capture. Delete the ratings folder to discard the captured ratings. The source and Lightroom's previews were never written.

Limits. A Smart Preview stand-in carries reduced raw data, not the original; exports from it are previews, not masters. Bringing captured ratings back into a Lightroom catalog must go through Lightroom's own writer, and this cookbook has no qualified procedure for that yet. Do not write the catalog externally, and do not copy the captured sidecars beside the originals and read them with **Read Metadata from Files**: that replaces the catalog's metadata from the sidecar, and a four-property sidecar is not the photo's full state. Label names and color classes also differ between applications. Until a procedure is qualified, keep the sidecars and journal as the record of the culling.

Reconcile catalog, XMP, ratings and dates

Prerequisites. Retain catalog develop/metadata fields, sidecar or embedded XMP, an untouched reference when available, and current capture/camera facts. Separate a metadata conflict badge from a rendering comparison. Read complete referenced settings, not just a nullable legacy adjustment flag.

Procedure. Compare the three representations field by field: current catalog, current source/sidecar, preserved reference. In two retained examples, the source DNGs lacked Camera Raw edits present in the catalog/reference while dozens of other scalars agreed. Camera-era file times versus later Lightroom-save reference times were consistent with an older/unflushed/restored source, but the writer was unproved. The badge alone did not identify that history.

The local PMRate integration writes a rating/label subset in sidecars, not the complete embedded DNG develop state. Its recorded introduction was after the operator's February incident, and a narrow process observation found no active writer. That excludes the newly introduced integration as the original February cause within the retained timeline; it does not identify every other writer or prove current absence indefinitely. Unrelated XMP rewrites can change modification ordering, so "newer" is not a semantic settings comparison.

Dates need the same discipline. Preserve original capture time, current catalog time, EXIF time, timezone/offset and actual corrected-date provenance separately. Filename dates come from ingestion and are not independent truth. A coherent offset can be evidence for a clock correction; several files disagreeing with a UTC-name hypothesis prevent a blanket timezone conclusion.

Verify. Compare parsed meaningful fields as well as raw bytes, preserving namespace-order-only differences, nil/absent fields and fractional/offset distinctions. Bind any accepted correction to complete current exposure/camera evidence and its exact intended cells. A same-exposure UID proves provenance, not raw-conversion equality. Keep source/preserved versions and settings/history protected.

Undo. Retain the pre-correction field manifest and normal Lightroom metadata-write/import history. Restore only the authorized scope through the normal UI/native owner; protect later edits before any family rollback. Do not mass-read stale sidecars into the catalog to clear conflict badges.

Limits. No current catalog-versus-XMP mismatch proves a specific historical writer. Do not infer a date's capture meaning from a settings panel's timestamp, file mtime, filename or a single scalar flag.

Start with supported performance maintenance

For ordinary performance trouble, begin with Adobe's guidance: adequate free space, catalog/previews on appropriate fast storage, intentional preview sizes, catalog optimization, and the relevant cache and Sync settings. Keep previews available during recovery rather than deleting them reflexively. Adobe: optimize performance.

The advanced recipes above answer observed mechanisms that routine maintenance did not explain. They are useful only when exact current evidence qualifies the mechanism. A measured API body, copy benchmark, native mapping transition, file publication, and server completion are different results. Report the one you actually verified.

A receipt you can reuse

A useful result says: what was requested, which actual sources/owners were bound, what was submitted once, the genuine terminal, expected native changes, preserved records/bytes, independently observed outcome, closed resources, and remaining holds. A failed result belongs in the same chain.

Worker capture cleanup must preserve that chain on ordinary failures too. The `CaptureFinalizer` component seals used sessions, or asks their producers to prove a never-used branch, and saves both inventories before closing the caller-owned directory descriptors once. An incomplete attempt retains its session and original failure; it cannot be relabeled unused. Reporting failure after acknowledged closure is recorded separately from ownership uncertainty. The parent must still audit every helper before worker reap and reservation release. This component does not start or settle processes, publish previews, or authorize release. Its fault controls and disposable journal checks qualify cleanup ordering, not a production publisher. Keep the old worker disabled until its composed lifecycle and inherited-resource checks pass; undo a source-only integration by retaining the previous worker and its receipts, never replaying a failed publication.

Keep private run receipts locally. Public reports should replace user paths and catalog/cloud identifiers with role names and scoped counts. Publish your own source and synthetic controls; link primary documentation for supported product behavior. Do not redistribute decompiled proprietary bodies or signed credentials.

Status examples in this guide are qualified observations through **2026-10-12**, not a claim that the entire catalog/cloud recovery is complete. Read the README for the current public implementation and release limits.